

# CSC3210 - Quiz - File Descriptors, Pipes, and Message Queues

Name: Solution

Answer the questions in the spaces provided. Work is to be done on your own without a computer/phone. Justify your answers and show all work where appropriate.

1. (10 points) After a `fork()`, the parent and child processes have separate address spaces but share entries in the file descriptor table. Explain what “sharing entries” means in practice, including what happens to the file position pointer. Support your answer with an example (code is not required).

**Solution:** After `fork()`, the child inherits copies of the parent’s open file descriptor numbers, but both the parent’s and child’s file descriptor table entries point to the same underlying kernel object. This means they share the same file position pointer. They do not share user-space memory, but they do share the kernel-managed file state.

For example, if a parent process writes data to the file using the file descriptor, then the data will be written and the file position pointer will be advanced to the location in the file at the end of the data. The child process will see the file position pointer in its new location.

2. (10 points) What is a pipe, and how does it differ from a regular file in terms of where data is stored and how the file position pointer is managed?

**Solution:** A pipe is a unidirectional data channel managed entirely by the operating system. Data is stored and managed in operating system memory, not on disk. Unlike a file, the programmer does not manage the file position pointer. Data written to the write end comes out the read end in order, and once read, it is consumed and gone. The OS handles all internal positioning.

# CSC3210 - Quiz - File Descriptors, Pipes, and Message Queues

3. (10 points) When using a pipe with multiple processes created with fork, why is it a good practice for processes to close the ends of the pipe when they are no longer needed? Support your answer with an example (code is not required).

**Solution:** Closing the ends of the pipe tells the operating system that the process no longer needs to use the pipe (for either reading or writing depending on which end is closed).

For example, when all write ends of the pipe are closed, the operating system knows that it is not possible for any more data to be written to the pipe. At that point, the operating system can send end of file to any process reading on the pipe.

If all processes close all ends of the pipe (both reading and writing), then the operating system knows it can release any storage allocated for the pipe.

4. (10 points) List two (2) ways that POSIX message queues differ from pipes.

**Solution:**

1. Message queues are structured: messages must be sent and received as complete, fixed-size units (atomic operations), while pipes are unstructured byte streams.
2. Message queues are named and persistent: they survive process termination and are identified by a string name, while anonymous pipes are temporary.
3. Message queues support message priorities: higher-priority messages are delivered first, while pipes deliver data strictly in FIFO byte order.

# CSC3210 - Quiz - OS Motivation and Computer Architecture

Name: Solution

Answer the questions in the spaces provided. Work is to be done on your own without a computer/phone. Justify your answers and show all work where appropriate.

1. (10 points) What is an abstraction in the context of operating systems? Support your answer with a specific example.

**Solution:** An abstraction in operating systems is a way to simplify complex hardware and software interactions, allowing programmers to work without needing to manage low-level details.

For example, file systems are abstraction provided by the OS that makes sharing and accessing persistent media (hard drives, solid state disks, etc.) easier.

2. (10 points) What is isolation in the context of multiple processes in computing? Why is isolation of processes important? Support your answer with a specific example.

**Solution:** Isolation is the separation of one process/application from another to protect memory and other allocated resources. Isolation is important to ensure that different process/applications running on the same system do not interfere with each other.

For example, a music player should not be able to access the credit card information entered into a web browser.

3. (10 points) What is multiprogramming? Why is it beneficial? Support your answer with a specific example.

**Solution:** Multiprogramming is the ability to run multiple programs simultaneously on a computer. It is beneficial because it improves hardware utilization by allowing multiple tasks to share CPU time, making the system more efficient.

For example, allowing a web browser to run multiple tabs at the same time allows multiple websites to be loaded at approximately the same time.

# CSC3210 - Quiz - OS Motivation and Computer Architecture

4. (10 points) Describe the fetch-execute cycle in a CPU? What steps are involved? What does the CPU do at each step?

**Solution:** The fetch-execute cycle involves fetching an instruction from memory, decoding it, executing it, and then writing back the result. This cycle is repeated continuously by the CPU.

1. Fetch - using the address contained in the program counter, the CPU loads the next instruction from memory
2. Decode - the CPU control unit decodes the instruction to determine what operation needs to be performed
3. Execute - the CPU control unit signals the appropriate hardware in the CPU to perform the necessary operation
4. Write-back - the result is written back either to memory, a register, the program counter, etc. based on the action performed

# CSC3210 - Quiz - System Calls and Processes

Name: Solution

Answer the questions in the spaces provided. Work is to be done on your own without a computer/phone. Justify your answers and show all work where appropriate.

1. (10 points) What is the difference between a program and a process? Provide an example of how multiple processes can be created from the same program.

**Solution:** A program is a static file containing code, while a process is a program in execution, which has a life cycle and changes its data during execution.

For example:

- the executable for the 'Word' word processor is an example of a program.
- multiple process instances of the 'Word' program can all be running from the same program each working on their own different document.

2. (10 points) What is the difference between user mode (space) and kernel/system mode (space)? Why would it not be a good idea to allow a user program to execute in kernel/system mode?

**Solution:** The difference is in CPU instruction privilege. The CPU is not able to execute certain instructions in user mode which restricts applications from accessing certain hardware features and memory address ranges. In kernel/system mode, the CPU can execute all instructions and has full access to all hardware and memory. A user program with access to all hardware and memory can access other processes running on the system and the OS itself which could cause a security risk or a safety risk.

# CSC3210 - Quiz - System Calls and Processes

3. (10 points) What hardware-supported mechanism allows a process to safely transition from user mode to kernel mode when making a system call? Describe how it ensures protection.

**Solution:** The mechanism used to safely switch from user mode to system mode is called a trap, which is a software interrupt that invokes the operating system code to handle the request and change the mode. It ensures protection because user code cannot switch to kernel mode on its own; only specific CPU instructions can change privilege levels. The entry addresses for system calls are stored in memory only the kernel can modify.

4. (10 points) In the 5 process state model one of the states is “waiting”. Describe the “waiting” state and provide one advantage of having waiting in addition to the other states. Why would a process be in a state of “waiting”?

**Solution:** The waiting state is the state a process enters when it cannot continue executing until some event occurs. A process in the waiting state is not eligible to run because it is waiting for something external. Using this state prevents the operating system from attempting to run a process that will not do any useful work when it is waiting for something (I/O, another process, etc.).